

134

表达式由一个或多个**运算对象**(operand)组成,对表达式求值得到一个**结果**(result)。字面值和变量是最简单的**表达式**(expression),其结果就是字面值和变量的值。把一个**运算符**(operator)和一个或多个运算对象组合起来可以生成较复杂的表达式。

4.1 基础

有几个基础概念对表达式的求值过程有影响,它们涉及大多数(甚至全部)表达式。本节先简要介绍这几个概念,后面的小节将做更详细的讨论。



4.1.1 基本概念

C++定义了一元运算符(unary operator)和二元运算符(binary operator)。作用于一个运算对象的运算符是一元运算符,如取地址符(&)和解引用符(*);作用于两个运算对象的运算符是二元运算符,如相等运算符(==)和乘法运算符(*)。除此之外,还有一个作用于三个运算对象的三元运算符。函数调用也是一种特殊的运算符,它对运算对象的数量没有限制。

一些符号既能作为一元运算符也能作为二元运算符。以符号*为例,作为一元运算符时执行解引用操作,作为二元运算符时执行乘法操作。一个符号到底是一元运算符还是二元运算符由它的上下文决定。对于这类符号来说,它的两种用法互不相干,完全可以当成两个不同的符号。

组合运算符和运算对象

对于含有多个运算符的复杂表达式来说,要想理解它的含义首先要理解运算符的优先级(precedence)、结合律(associativity)以及运算对象的求值顺序(order of evaluation)。例如,下面这条表达式的求值结果依赖于表达式中运算符和运算对象的组合方式:

```
5 + 10 * 20/2;
```

乘法运算符(*)是一个二元运算符,它的运算对象有4种可能:10和20、10和20/2、15和20、15和20/2。下一节将介绍如何理解这样一条表达式。

运算对象转换

在表达式求值的过程中,运算对象常常由一种类型转换成另外一种类型。例如,尽管一般的二元运算符都要求两个运算对象的类型相同,但是很多时候即使运算对象的类型不同也没有关系,只要它们能被转换(参见2.1.2节,第32页)成同一种类型即可。

类型转换的规则虽然有点复杂,但大多数都合乎情理、容易理解。例如,整数能转换成浮点数,浮点数也能转换成整数,但是指针不能转换成浮点数。让人稍微有点意外的是,小整数类型(如bool、char、short等)通常会被**提升**(promoted)成较大的整数类型,主要是int。4.11节(第141页)将详细介绍类型转换的细节。

135

重载运算符

C++语言定义了运算符作用于内置类型和复合类型的运算对象时所执行的操作。当运算符作用于类类型的运算对象时,用户可以自行定义其含义。因为这种自定义的过程事实上是为已存在的运算符赋予了另外一层含义,所以称之为**重载运算符**(overloaded operator)。IO库的>>和<<运算符以及string对象、vector对象和迭代器使用的运算

符都是重载的运算符。

我们使用重载运算符时，其包括运算对象的类型和返回值的类型，都是由该运算符定义的；但是运算对象的个数、运算符的优先级和结合律都是无法改变的。

左值和右值



C++的表达式要不然是**右值**（rvalue，读作“are-value”），要不然就是**左值**（lvalue，读作“ell-value”）。这两个名词是从C语言继承过来的，原本是为了帮助记忆：左值可以位于赋值语句的左侧，右值则不能。

在C++语言中，二者的区别就没那么简单了。一个左值表达式的求值结果是一个对象或者一个函数，然而以常量对象为代表的某些左值实际上不能作为赋值语句的左侧运算对象。此外，虽然某些表达式的求值结果是对象，但它们是右值而非左值。可以做一个简单的归纳：当一个对象被用作右值的时候，用的是对象的值（内容）；当对象被用作左值的时候，用的是对象的身份（在内存中的位置）。

不同的运算符对运算对象的要求各不相同，有的需要左值运算对象、有的需要右值运算对象；返回值也有差异，有的得到左值结果、有的得到右值结果。一个重要的原则（参见13.6节，第470页将介绍一种例外的情况）是在需要右值的地方可以用左值来代替，但是不能把右值当成左值（也就是位置）使用。当一个左值被当成右值使用时，实际使用的是它的内容（值）。到目前为止，已经有几种我们熟悉的运算符是要用到左值的。

- 赋值运算符需要一个（非常量）左值作为其左侧运算对象，得到的结果也仍然是一个左值。
- 取地址符（参见2.3.2节，第47页）作用于一个左值运算对象，返回一个指向该运算对象的指针，这个指针是一个右值。
- 内置解引用运算符、下标运算符（参见2.3.2节，第48页；参见3.5.2节，第104页）、迭代器解引用运算符、string和vector的下标运算符（参见3.4.1节，第95页；参见3.2.3节，第83页；参见3.3.3节，第91页）的求值结果都是左值。
- 内置类型和迭代器的递增递减运算符（参见1.4.1节，第11页；参见3.4.1节，第96页）作用于左值运算对象，其前置版本（本书之前章节所用的形式）所得的结果也是左值。

接下来在介绍运算符的时候，我们将会注明该运算符的运算对象是否必须是左值以及其求值结果是否是左值。

使用关键字decltype（参见2.5.3节，第62页）的时候，左值和右值也有所不同。如果表达式的求值结果是左值，decltype作用于该表达式（不是变量）得到一个引用类型。举个例子，假定p的类型是int*，因为解引用运算符生成左值，所以decltype(*p)的结果是int&。另一方面，因为取地址运算符生成右值，所以decltype(&p)的结果是int**，也就是说，结果是一个指向整型指针的指针。

136

4.1.2 优先级与结合律



复合表达式（compound expression）是指含有两个或多个运算符的表达式。求复合表达式的值需要首先将运算符和运算对象合理地组合在一起，优先级与结合律决定了运算对象组合的方式。也就是说，它们决定了表达式中每个运算符对应的运算对象来自表达式的哪一部分。表达式中的括号无视上述规则，程序员可以使用括号将表达式的某个局部括起来使其得到优先运算。

一般来说,表达式最终的值依赖于其子表达式的组合方式。高优先级运算符的运算对象要比低优先级运算符的运算对象更为紧密地组合在一起。如果优先级相同,则其组合规则由结合律确定。例如,乘法和除法的优先级相同且都高于加法的优先级。因此,乘法和除法的运算对象会首先组合在一起,然后才能轮到加法和减法的运算对象。算术运算符满足左结合律,意味着如果运算符的优先级相同,将按照从左向右的顺序组合运算对象:

- 根据运算符的优先级,表达式 $3+4*5$ 的值是 23,不是 35。
- 根据运算符的结合律,表达式 $20-15-3$ 的值是 2,不是 8。

举一个稍微复杂一点的例子,如果完全按照从左向右的顺序求值,下面的表达式将得到 20:

```
6 + 3 * 4 / 2 + 2
```

也有一些人会计算得到 9、14 或者 36,然而在 C++ 语言中真实的计算结果应该是 14。这是因为这条表达式事实上与下述表达式等价:

```
// 这条表达式中的括号符合默认的优先级和结合律
((6 + ((3 * 4) / 2)) + 2)
```

括号无视优先级与结合律

括号无视普通的组合规则,表达式中括号括起来的部分被当成一个单元来求值,然后再与其他部分一起按照优先级组合。例如,对上面这条表达式按照不同方式加上括号就能得到 4 种不同的结果:

```
// 不同的括号组合导致不同的组合结果
cout << (6 + 3) * (4 / 2 + 2) << endl;      // 输出 36
cout << ((6 + 3) * 4) / 2 + 2 << endl;      // 输出 20
cout << 6 + 3 * 4 / (2 + 2) << endl;        // 输出 9
```



优先级与结合律有何影响

137

由前面的例子可以看出,优先级会影响程序的正确性,这一点在 3.5.3 节(第 107 页)介绍的解引用和指针运算中也有所体现:

```
int ia[] = {0,2,4,6,8}; // 含有 5 个整数的数组
int last = *(ia + 4);    // 把 last 初始化成 8,也就是 ia[4] 的值
last = *ia + 4;         // last = 4,等价于 ia[0] + 4
```

如果想访问 $ia+4$ 位置的元素,那么加法运算两端的括号必不可少。一旦去掉这对括号, $*ia$ 就会首先组合在一起,然后 4 再与 $*ia$ 的值相加。

结合律对表达式产生影响的一个典型示例是输入输出运算,4.8 节(第 138 页)将要介绍 IO 相关的运算符满足左结合律。这一规则意味着我们可以把几个 IO 运算组合在一条表达式当中:

```
cin >> v1 >> v2;      // 先读入 v1,再读入 v2
```

4.12 节(第 147 页)罗列出了全部的运算符,并用双横线将它们分割成若干组。同一组内的运算符优先级相同,组的位置越靠前组内的运算符优先级越高。例如,前置递增运算符和解引用运算符的优先级相同并且都比算术运算符的优先级高。表中同样列出了每个运算符在哪一页有详细的描述,有些运算符之前已经使用过了,大多数运算符的细节将在本章剩余部分逐一介绍,还有几个运算符将在后面的内容中提及。

4.1.2 节练习

练习 4.1: 表达式 $5+10*20/2$ 的求值结果是多少?

练习 4.2: 根据 4.12 节中的表, 在下述表达式的合理位置添加括号, 使得添加括号后运算对象的组合顺序与添加括号前一致。

(a) `*vec.begin()` (b) `*vec.begin() + 1`

4.1.3 求值顺序



优先级规定了运算对象的组合方式, 但是没有说明运算对象按照什么顺序求值。在大多数情况下, 不会明确指定求值的顺序。对于如下的表达式

```
int i = f1() * f2();
```

我们知道 `f1` 和 `f2` 一定会在执行乘法之前被调用, 因为毕竞相乘的是这两个函数的返回值。但是我们无法知道到底 `f1` 在 `f2` 之前调用还是 `f2` 在 `f1` 之前调用。

对于那些没有指定执行顺序的运算符来说, 如果表达式指向并修改了同一个对象, 将会引发错误并产生未定义的行为 (参见 2.1.2 节, 第 33 页)。举个简单的例子, `<<` 运算符没有明确规定何时以及如何对运算对象求值, 因此下面的输出表达式是未定义的:

138

```
int i = 0;
cout << i << " " << ++i << endl; // 未定义的
```

因为程序是未定义的, 所以我们无法推断它的行为。编译器可能先求 `++i` 的值再求 `i` 的值, 此时输出结果是 `1 1`; 也可能先求 `i` 的值再求 `++i` 的值, 输出结果是 `0 1`; 甚至编译器还可能做完全不同的操作。因为此表达式的行为不可预知, 因此不论编译器生成什么样的代码程序都是错误的。

有 4 种运算符明确规定了运算对象的求值顺序。第一种是 3.2.3 节 (第 85 页) 提到的逻辑与 (`&&`) 运算符, 它规定先求左侧运算对象的值, 只有当左侧运算对象的值为真时才继续求右侧运算对象的值。另外三种分别是逻辑或 (`||`) 运算符 (参见 4.3 节, 第 126 页)、条件 (`?:`) 运算符 (参见 4.7 节, 第 134 页) 和逗号 (`,`) 运算符 (参见 4.10 节, 第 140 页)。

求值顺序、优先级、结合律



运算对象的求值顺序与优先级和结合律无关, 在一条形如 `f() + g() * h() + j()` 的表达式中:

- 优先级规定, `g()` 的返回值和 `h()` 的返回值相乘。
- 结合律规定, `f()` 的返回值先与 `g()` 和 `h()` 的乘积相加, 所得结果再与 `j()` 的返回值相加。
- 对于这些函数的调用顺序没有明确规定。

如果 `f`、`g`、`h` 和 `j` 是无关函数, 它们既不会改变同一对象的状态也不执行 IO 任务, 那么函数的调用顺序不受限制。反之, 如果其中某几个函数影响同一对象, 则它是一条错误的表达式, 将产生未定义的行为。

建议: 处理复合表达式

139

以下两条经验准则对书写复合表达式有益:

1. 拿不准的时候最好用括号来强制让表达式的组合关系符合程序逻辑的要求。
2. 如果改变了某个运算对象的值，在表达式的其他地方不要再使用这个运算对象。
- 第 2 条规则有一个重要例外，当改变运算对象的子表达式本身就是另外一个子表达式的运算对象时该规则无效。例如，在表达式*++iter 中，递增运算符改变 iter 的值，iter（已经改变）的值又是解引用运算符的运算对象。此时（或类似的情况下），求值的顺序不会成为问题，因为递增运算（即改变运算对象的子表达式）必须先求值，然后才轮到解引用运算。显然，这是一种很常见的用法，不会造成什么问题。

4.1.3 节练习

练习 4.3: C++语言没有明确规定大多数二元运算符的求值顺序，给编译器优化留下了余地。这种策略实际上是在代码生成效率和程序潜在缺陷之间进行了权衡，你认为这可以接受吗？请说出你的理由。

4.2 算术运算符

表 4.1: 算术运算符（左结合律）		
运算符	功能	用法
+	一元正号	+ expr
-	一元负号	- expr
*	乘法	expr * expr
/	除法	expr / expr
%	求余	expr % expr
+	加法	expr + expr
-	减法	expr - expr

表 4.1（以及后面章节的运算符表）按照运算符的优先级将其分组。一元运算符的优先级最高，接下来是乘法和除法，优先级最低的是加法和减法。优先级高的运算符比优先级低的运算符组合得更紧密。上面的所有运算符都满足左结合律，意味着当优先级相同时按照从左向右的顺序进行组合。

除非另做特殊说明，算术运算符都能作用于任意算术类型（参见 2.1.1 节，第 30 页）以及任意能转换为算术类型的类型。算术运算符的运算对象和求值结果都是右值。如 4.11 节（第 141 页）描述的那样，在表达式求值之前，小整数类型的运算对象被提升成较大的整数类型，所有运算对象最终会转换成同一类型。

140

一元正号运算符、加法运算符和减法运算符都能作用于指针。3.5.3 节（第 106 页）已经介绍过二元加法和减法运算符作用于指针的情况。当一元正号运算符作用于一个指针或者算术值时，返回运算对象值的一个（提升后的）副本。

一元负号运算符对运算对象值取负后，返回其（提升后的）副本：

```
int i = 1024;
int k = -i;      // k 是-1024
bool b = true;
bool b2 = -b;    // b2 是 true!
```